

Weak evaluation strategies in the λ -calculus

Pablo Barenbaum

2018-05-10

In this note we review three basic weak evaluation strategies: *call-by-name*, *call-by-value*, and *call-by-need*, by means of three formal presentations: *small-step evaluation*, *big-step evaluation* and an *abstract machine*. We sketch how to prove that the three presentations are equivalent in the case of the call-by-name strategy. For the other two strategies similar techniques may be used (although the proofs are much more complex, especially in the call-by-need case).

Contents

1	Call-by-name	1
1.1	Small-step evaluation	1
1.2	Big-step evaluation	2
1.3	Abstract machine	2
1.4	Equivalence: big-step evaluation vs. abstract machine	4
1.5	Equivalence: small-step evaluation vs. abstract machine	4
2	Call-by-value	5
2.1	Small-step evaluation	5
2.2	Big-step evaluation	5
2.3	Abstract machine	6
3	Call-by-need	6
3.1	Small-step evaluation	6
3.2	Big-step evaluation	7
3.3	Abstract machine	7

1 Call-by-name

1.1 Small-step evaluation

Definition 1 (Small-step call-by-name evaluation). Terms and evaluation contexts are given by:

Terms	t	$::=$	$x \mid \lambda x.t \mid tt$
Weak head contexts	H	$::=$	$\square \mid Ht$

The binary relation of small-step call-by-name evaluation is defined as follows:

$$H\langle(\lambda x.t) s\rangle \rightarrow_{\text{name}} H\langle t\{x := s\}\rangle$$

Remark 2 (Determinism). If $t \rightarrow_{\text{name}} t_1$ and $t \rightarrow_{\text{name}} t_2$ then $t_1 = t_2$.

Exercise 3. Evaluate $(\lambda x.xx)(II)$ using small-step call-by-name evaluation.

1.2 Big-step evaluation

Definition 4 (Big-step call-by-name evaluation). Call-by-name environments and closures are given by the following abstract syntax:

$$\begin{array}{ll} \text{Environments } e & ::= \bullet \mid [x \mapsto c]:e \\ \text{Closures } c & ::= (t, e) \end{array}$$

Environment concatenation is written $e_1 : e_2$. We also write $\text{dom } e$ for the set of variables defined in e , and $e(x)$ for the value of x in e .

The *big-step call-by-name evaluation* judgment $t \Downarrow^e c$, relates a source term t , an environment e mapping variables to closures, and the resulting closure c . Derivable judgments are inductively defined as follows:

$$\frac{e(x) = (t, e') \quad t \Downarrow^{e'} c}{x \Downarrow^e c} \quad \frac{}{\lambda x.t \Downarrow^e (\lambda x.t, e)} \quad \frac{t \Downarrow^e (\lambda x.t', e') \quad t' \Downarrow^{[x \mapsto (s, e)]:e'} c}{t s \Downarrow^e c}$$

Remark 5. If $t \Downarrow^e c$ holds, then c is of the form $(\lambda x.s, e)$.

Exercise 6. Evaluate $(\lambda x.xx)(II)$ using big-step call-by-name evaluation.

Algorithm 7. A big-step call-by-name evaluator in Haskell:

```

type Id      = String
data Term    = Var Id | Lam Id Term | App Term Term
type Env     = [(Id, Closure)]
data Closure = C Term Env

eval :: Term -> Env -> Closure
eval (Var x) e = let Just (C t e') = lookup x e in
                  eval t e'
eval (Lam x t) e = C (Lam x t) e
eval (App t s) e = let C (Lam x t') e' = eval t e in
                   eval t' ((x, C s e) : e')

```

1.3 Abstract machine

Definition 8 (Krivine Abstract Machine — KAM). Syntax:

$$\begin{array}{ll} \text{States } S & ::= \langle t \mid e \mid \pi \rangle \\ \text{Stacks } \pi & ::= \bullet \mid c:\pi \end{array}$$

The transition relation is defined as $\mapsto \stackrel{\text{def}}{=} \mapsto_{\text{app}} \cup \mapsto_{\text{lam}} \cup \mapsto_{\text{var}}$, where:

$$\begin{array}{lll} \langle t \ s \mid e \mid \pi \rangle & \mapsto_{\text{app}} & \langle t \mid e \mid (s, e) : \pi \rangle \\ \langle \lambda x. t \mid e \mid c : \pi \rangle & \mapsto_{\text{lam}} & \langle t \mid [x \mapsto c] : e \mid \pi \rangle \\ \langle x \mid e \mid \pi \rangle & \mapsto_{\text{var}} & \langle t \mid e' \mid \pi \rangle \quad \text{if } e(x) = (t, e') \end{array}$$

Stack concatenation is denoted by $\pi_1 : \pi_2$.

Remark 9 (Determinism). If $S \mapsto S_1$ and $S \mapsto S_2$ then $S_1 = S_2$.

Exercise 10. Evaluate $(\lambda x. xx)(II)$ in the KAM.

Algorithm 11. An implementation of the KAM in Haskell:

```

type Id      = String
data State   = S Term Stack Env
data Term    = Var Id | Lam Id Term | App Term Term
type Env     = [(Id, Closure)]
data Closure = C Term Env
type Stack   = [Closure]

exec1 :: State -> Maybe State
exec1 (S (App t s) p e)      = Just (S t (C s e : p) e)
exec1 (S (Lam x t) (c : p) e) = Just (S t p ((x, c) : e))
exec1 (S (Var x) p e)        = let Just (C t e') = lookup x e
                               in Just (S t p e')
exec1 _                      = Nothing

exec :: State -> State
exec s = case exec1 s of
  Nothing -> s
  Just s'  -> exec s'

```

Definition 12. The notion of being *closed* is defined inductively as follows:

- A term t is closed in an environment e if $\text{fv}(t) \subseteq \text{dom } e$. A term t is closed (*a secas*) if $\text{fv}(t) = \emptyset$.
- A closure (t, e) is closed if the term t is closed in e and, moreover, e is closed.
- An environment $[x_1 \mapsto c_1] : \dots : [x_n \mapsto c_n]$ is closed if c_i is closed for all $i = 1..n$.
- A stack $c_1 : \dots : c_n$ is closed if c_i is closed for all $i = 1..n$.
- A state $\langle t \mid e \mid \pi \rangle$ is closed if the closure (t, e) is closed and the stack π is closed.

Lemma 13 (KAM invariant). *If $S \mapsto S'$ and S is closed then S' is closed. Remark that if t is a closed term, then $\langle t \mid \bullet \mid \bullet \rangle$ trivially fulfills the invariant.*

Proof. By case analysis on the transitions. ■

□

1.4 Equivalence: big-step evaluation vs. abstract machine

Lemma 14 (Stack weakening). *If $\langle t \mid e \mid \pi_1 \rangle \mapsto \langle t' \mid e' \mid \pi'_1 \rangle$ then $\langle t \mid e \mid \pi_1 : \pi_2 \rangle \mapsto \langle t' \mid e' \mid \pi'_1 : \pi_2 \rangle$.*

Proof. By case analysis on the transitions of the KAM. $\square$

Proposition 15 (From big-step to the KAM). *If $t \Downarrow^e (s, e')$ then $\langle t \mid e \mid \bullet \rangle \mapsto^* \langle s \mid e' \mid \bullet \rangle$.*

Proof. By induction on the derivation that $t \Downarrow^e (s, e')$ using Lem. 14. $\square$

As an intermediate step to prove the converse of Prop. 15, we need the following generalization of the judgment $t \Downarrow^e c$:

Definition 16 (Generalized evaluation). Big-step call-by-name evaluation is generalized for an arbitrary stack π as follows:

$$\frac{t \Downarrow^e c}{t \Downarrow_{\pi}^e c} \quad \frac{t \Downarrow^e (\lambda x. t', e') \quad t' \Downarrow_{\pi}^{[x \mapsto c_1] : e'} c_2}{t \Downarrow_{c_1 : \pi}^e c_2}$$

Lemma 17 (Properties of generalized evaluation). *The judgment $t \Downarrow_{\pi}^e c$ has the following properties:*

1. *If $e(x) = (t, e')$ and $t \Downarrow_{\pi}^{e'} c$ then $x \Downarrow_{\pi}^e c$.*
2. *If $t \Downarrow^e (\lambda x. t', e')$ and $t' \Downarrow_{\pi}^{[x \mapsto (s, e)] : e'} c$ then $ts \Downarrow_{\pi}^e c$.*

Proof. By induction on π . $\square$

Proposition 18 (From KAM to big-step). *If $S = \langle t \mid e \mid \pi \rangle \mapsto^* \langle s \mid e' \mid \pi' \rangle = S'$ and S fulfills the KAM invariant and S' is in $\mapsto$ -normal form, then π' is empty and $t \Downarrow_{\pi}^e (s, e')$.*

Proof. By induction on the number of transitions in $S \mapsto^* S'$ and case analysis on the shape of t , relying on Lem. 17. $\square$

1.5 Equivalence: small-step evaluation vs. abstract machine

Definition 19 (KAM decoding).

$$\begin{aligned} \frac{\langle t \mid e \mid \pi \rangle}{\bullet} &\stackrel{\text{def}}{=} \frac{\pi \langle t^e \rangle}{\square} \\ \frac{c : \pi}{t^{\bullet}} &\stackrel{\text{def}}{=} \frac{\pi \langle \square c \rangle}{t} \\ \frac{t^{[x \mapsto c] : e}}{t^{\bullet}} &\stackrel{\text{def}}{=} t \{ x := \underline{c} \}^e \end{aligned}$$

Lemma 20 (Properties of the decoding). *The KAM decoding has the following properties:*

1. $(\lambda x. t)^e = \lambda x. t^e$
2. $(t s)^e = t^e s^e$
3. *If e and e' are closed environments equal up to a permutation then $t^e = t^{e'}$.*
4. *If t is a closed term, then $t^e = t$.*

Proof. By induction on e . $\square$

Proposition 21 (KAM correctness). *If $S \mapsto S'$ and the states fulfill the KAM invariant then $\underline{S} \rightarrow_{\text{name}}^* \underline{S}'$. Furthermore:*

- *If $S \mapsto_{\text{app}} S'$ then $\underline{S} = \underline{S}'$.*
- *If $S \mapsto_{\text{lam}} S'$ then $\underline{S} \rightarrow_{\text{name}} \underline{S}'$.*
- *If $S \mapsto_{\text{var}} S'$ then $\underline{S} = \underline{S}'$.*

Proof. By case analysis on the transitions of the KAM, using Lem. 20. $\square$

Proposition 22 (KAM completeness). *If $t \rightarrow_{\text{name}} t'$ and S is a state fulfilling the KAM invariant such that $\underline{S} = t$, then there exists a state S' such that $S \mapsto^* S'$ and $\underline{S}' = t'$.*

Proof. Observe that the $\mapsto_{\text{var}}$ transition strictly decreases the size of the environment, and the $\mapsto_{\text{app}}$ transition preserves the size of the environment while strictly decreasing the size of the term. Hence $\mapsto_{\text{app, var}}$ is terminating.

Normalize S with respect to $\mapsto_{\text{app, var}}$ transitions, obtaining $S \mapsto^* S_1$. By correctness (Prop. 21), $t = \underline{S} = \underline{S}_1$. Note that the term of S_1 is an abstraction, so $S_1 = \langle \lambda x.s \mid \pi \mid e \rangle$. If the stack π is empty, then $t = \underline{S}_1 = (\lambda x.s)^e$ is in $\rightarrow_{\text{name}}$ -normal form contradicting the fact that $t \rightarrow_{\text{name}} t'$. So the stack is non-empty, $\pi = c:\pi'$ and we have:

$$S \mapsto_{\text{app, var}}^* S_1 = \langle \lambda x.s \mid c:\pi' \mid e \rangle \mapsto_{\text{lam}} \langle s \mid \pi' \mid [x \mapsto c]:e \rangle = S'$$

By correctness (Prop. 21), $t = \underline{S} = \underline{S}_1 \rightarrow_{\text{name}} \underline{S}'$. So by determinism of both $\mapsto$ and $\rightarrow_{\text{name}}$ we conclude that $\underline{S}' = t'$, as required. $\square$

2 Call-by-value

2.1 Small-step evaluation

Definition 23 (Small-step call-by-value evaluation). Terms and evaluation contexts are given by:

Terms	$t ::= x \mid \lambda x.t \mid tt$
Values	$v ::= \lambda x.t$
Weak by-value contexts	$V ::= \square \mid Vt \mid vV$

The binary relation of small-step call-by-value evaluation is defined as follows:

$$V\langle \lambda x.t \rangle v \rightarrow_{\text{value}} V\langle t \{x := v\} \rangle$$

Exercise 24. Evaluate $(\lambda x.xx)(II)$ using small-step call-by-value evaluation.

2.2 Big-step evaluation

Definition 25 (Big-step call-by-value evaluation). Call-by-value environments and closures are given by the following abstract syntax:

Environments	$e ::= \bullet \mid [x \mapsto c]:e$
Closures	$c ::= (v, e)$

Derivability of the *big-step call-by-value evaluation* judgment $t \Downarrow^e c$ is defined as follows:

$$\frac{e(x) = c}{x \Downarrow^e c} \quad \frac{}{\lambda x. t \Downarrow^e (\lambda x. t, e)} \quad \frac{t \Downarrow^e (\lambda x. t', e') \quad s \Downarrow^e c_1 \quad t' \Downarrow^{[x \mapsto c]: e'} c_2}{t s \Downarrow^e c_2}$$

Exercise 26. Evaluate $(\lambda x. xx)(II)$ using big-step call-by-value evaluation.

2.3 Abstract machine

Definition 27 (CEK Machine). Syntax:

$$\begin{aligned} \text{States } S &::= \langle t \mid e \mid \pi \rangle \\ \text{Stacks } \pi &::= \bullet \mid \mathbf{A}(t, e) : \pi \mid \mathbf{F}(v, e) : \pi \end{aligned}$$

The transition relation is defined as follows:

$$\begin{aligned} \langle t s \mid e \mid \pi \rangle &\mapsto \langle t \mid e \mid \mathbf{F}(s, e) : \pi \rangle \\ \langle v \mid e \mid \mathbf{A}(t, e') : \pi \rangle &\mapsto \langle t \mid e' \mid \mathbf{F}(v, e) : \pi \rangle \\ \langle v \mid e \mid \mathbf{F}(\lambda x. t, e') : \pi \rangle &\mapsto \langle t \mid [x \mapsto (v, e)] : e' \mid \pi \rangle \\ \langle x \mid e \mid \pi \rangle &\mapsto \langle v \mid e' \mid \pi \rangle \quad \text{if } e(x) = (v, e') \end{aligned}$$

Exercise 28. Evaluate $(\lambda x. xx)(II)$ in the CEK.

3 Call-by-need

3.1 Small-step evaluation

In contrast to call-by-name and call-by-value, small-step call-by-need evaluation cannot be expressed directly in the λ -calculus. To be able to express call-by-need as a small-step reduction strategy, we need to extend the set of terms with *explicit substitutions*.

Definition 29 (Small-step call-by-need evaluation). Terms and evaluation contexts are given by:

$$\begin{aligned} \text{Terms } t &::= x \mid \lambda x. t \mid t t \mid t[x := t] \\ \text{Substitution contexts } L &::= \square \mid L[x := t] \\ \text{Values } v &::= \lambda x. t \\ \text{Weak by-need contexts } N &::= \square \mid N t \mid N[x := t] \mid N\langle x \rangle[x := N] \end{aligned}$$

Substitution contexts are lists of explicit substitutions, $L = \square[x_1 := t_1] \dots [x_n := t_n]$. We write tL for $t[x_1 := t_1] \dots [x_n := t_n]$ rather than $L\langle t \rangle$. The binary relation of small-step call-by-need evaluation is defined as the union $\rightarrow_{\text{need}} = \rightarrow_{\text{db}} \cup \rightarrow_{1v} \cup \rightarrow_{\text{gc}}$ of the following three relations:

$$\begin{aligned} N\langle \langle \lambda x. t \rangle L s \rangle &\rightarrow_{\text{db}} N\langle t[x := s] L \rangle && \text{distant beta} \\ N_1\langle N_2\langle x \rangle[x := vL] \rangle &\rightarrow_{1v} N_1\langle N_2\langle v \rangle[x := v]L \rangle && \text{linear value substitution} \\ N\langle t[x := s] \rangle &\rightarrow_{\text{gc}} N\langle t \rangle && \text{if } x \notin \text{fv}(t) \quad \text{garbage collection} \end{aligned}$$

The three rules above are not deterministic. For example, in a term like $(II)[x := t]$ the first and the third rule may apply. One can show that the system without the last rule is deterministic, and the garbage collection rule can be postponed:

Lemma 30 (Postponement of garbage-collection). *If $t \rightarrow_{gc} \rightarrow_{db,lv} s$ then $t \rightarrow_{db,lv}^* s$.*

Proof. By case analysis on all the possibilities in which a $\rightarrow_{gc}$ step is followed by a $\rightarrow_{db,lv}$ step. ■ □

Exercise 31. Evaluate $(\lambda x.xx)(II)$ using small-step call-by-need evaluation.

3.2 Big-step evaluation

Definition 32 (Big-step call-by-need evaluation). Let $\mathcal{L} = \{\ell_1, \ell_2, \dots\}$ be a denumerable set of *memory locations*. Call-by-need environments and closures are given by the following abstract syntax:

Environments	e	$::=$	$\bullet \mid [x \mapsto \ell]:e$
Memories	μ	$::=$	$\bullet \mid [\ell \mapsto \mathbf{T}(t, e)]:\mu \mid [\ell \mapsto \mathbf{V}(v, e)]:\mu$
Closures	c	$::=$	(v, e)

Derivability of the *big-step call-by-need evaluation* judgment $t @ \mu_1 \Downarrow^e c @ \mu_2$ is defined as follows:

$$\begin{array}{c}
 \frac{\ell = e(x) \quad \mu(\ell) = \mathbf{V}(v, e')}{x @ \mu \Downarrow^e (v, e') @ \mu} \quad \frac{}{\lambda x.t @ \mu \Downarrow^e (\lambda x.t, e) @ \mu} \\
 \\
 \frac{\ell = e(x) \quad \mu_1(\ell) = \mathbf{T}(t, e') \quad t @ \mu_1 \Downarrow^{e'} (v, e'') @ \mu_2}{x @ \mu_1 \Downarrow^e (v, e'') @ [\ell \mapsto \mathbf{V}(v, e'')]:\mu_2} \\
 \\
 \frac{t @ \mu_1 \Downarrow^e (\lambda x.t', e') @ \mu_2 \quad \ell \text{ fresh} \quad t' @ [\ell \mapsto \mathbf{T}(s, e)]:\mu_2 \Downarrow^{[x \mapsto \ell]:e'} c @ \mu_3}{ts @ \mu_1 \Downarrow^e c @ \mu_3}
 \end{array}$$

Exercise 33. Evaluate $(\lambda x.xx)(II)$ using big-step call-by-need evaluation.

3.3 Abstract machine

The following machine is based on Sestoft's:

Definition 34 (Milner by-need Abstract Machine). Syntax:

States	S	$::=$	$\langle t \mid \pi \mid D \mid E \rangle$
Stacks	π	$::=$	$\bullet \mid t:\pi$
Dumps	D	$::=$	$(x, \pi):D$
Global environments	E	$::=$	$\bullet \mid [x \mapsto t]:E$

The transition relation is defined as follows:

$$\begin{array}{ll}
 \langle ts \mid \pi \mid D \mid E \rangle & \mapsto \langle t \mid s:\pi \mid D \mid E \rangle \\
 \langle \lambda x.t \mid s:\pi \mid D \mid E \rangle & \mapsto \langle t \mid \pi \mid D \mid [x \mapsto s]:E \rangle \\
 \langle x \mid \pi \mid D \mid E \rangle & \mapsto \langle t \mid \bullet \mid (x, \pi):D \mid E \rangle \quad \text{if } E(x) = t \\
 \langle v \mid \bullet \mid (x, \pi):D \mid E \rangle & \mapsto \langle v^\alpha \mid \pi \mid D \mid [x \mapsto v]:E \rangle
 \end{array}$$

Exercise 35. Evaluate $(\lambda x.xx)(II)$ in Milner by-need Abstract Machine.